

Smart Camera Meter OCR: An Exploration of Ultra-Lightweight End-to-End Sequence Recognition for Resource-Constrained Edge Devices

Israel Zimmerman, Reuven Simhayov

Data Science Course, Final Project, 2025–2026

ABSTRACT

This paper describes an attempt to build an ultra-lightweight on-device OCR system for analog utility meters, targeting deployment on an ESP32-C6 microcontroller (160 MHz RISC-V, 512 KB SRAM). We implemented a Convolutional Recurrent Neural Network (CRNN) with Connectionist Temporal Classification (CTC) loss, trained on 100,000 synthetic meter images and subsequently fine-tuned on 35 manually labelled real photographs. The model achieves 6.5% overall accuracy on real images after fine-tuning up from 0% before with night-condition images reaching 22.2%. These results reveal a substantial simulation-to-reality gap that 35 labelled examples cannot bridge. We document the full architecture, training challenges encountered (including a critical CTC SoftMax incompatibility and Keras API changes), and six implementation lessons. The primary conclusion is that the bottleneck in this project is not model design but labelled data: without 200–300 real annotated images, even a structurally sound model cannot generalize across varied lighting and imaging conditions.

Keywords: OCR, CRNN, CTC, edge inference, synthetic data, sim-to-real gap, meter reading

1. INTRODUCTION

The global push toward smart utility infrastructure has renewed interest in low-cost meter reading solutions. Most analog meters electricity, water, gas were never designed for automated reading. Replacing them entirely is prohibitively expensive at scale, but retrofitting them with a small clip-on camera cover module is increasingly feasible. An ESP32-C6-based clip-on device with an OV7676 with 850 nm filter grayscale camera can be produced for under twenty dollars, communicates over LoRaWAN with Cellular Smart Gateway communicate with hundreds Smart Meters local network to Cellular network. The Smart Camera Meter operates on Electric Meter using inductive energy harvesting from the power lines to the meter, without requiring an external power supply or a small battery that will last for years on Gas Meter or Water Meter. The appeal is obvious: if such a device with a small camera can accurately read the display do OCR between readings and transmit the reading value every 15 minutes, the economics of smart metering shift dramatically for remote or low-income areas. The infrastructure of the network is based on Central Cloud with local Smart Gateways and Multiple Smart Camera Meters installed on building or rural areas communicate on low bandwidth with LoRAWan (Up to 10Km).

We are using 3 OCR levels to achieve maximum reading accuracy:

1. Smart Camera Meter local OCR (This project)
2. Upon failure, local known OCR model
3. Upon failure, use Google vision OCR

The engineering challenge is severe. The ESP32-C6 runs at 160 MHz on a RISC-V core with approximately 512 KB of SRAM and no floating-point unit. The entire OCR pipeline, image capture, de-skewing (fixing Lens geometry), preprocessing, digit detection, sequence recognition, and decoding must complete in under 600 seconds on 400×300 grayscale frames (850 nm capture images on any time or weather). The model weights must fit in under 2 MB of flash memory. And the system must achieve over 95% full-string accuracy on meter readings that span 5 to 8 digits, under variable outdoor lighting and 850 nm leds, through glass covers that introduce reflections, and at angles that may not be precisely controlled during installation. Commercial cloud OCR solutions cannot be used: LoRaWAN's narrow bandwidth is intended for small telemetry packets, not image uploads.

Our original proposal envisioned a two-stage pipeline. A lightweight detection network (YOLOv8-nano or SSD-MobileNet-v2 pruned to under 300 KB) would first localize the digit region within the frame, and a second ultra-small classifier (EfficientNet-Lite0 or MobileNet-v3-Small quantized to INT8, under 1.2 MB) would then classify individual characters. During implementation, we pivoted to a single end-to-end CRNN+CTC architecture, which eliminates the need for character-level segmentation entirely. Segmentation-free sequence recognition is both more elegant and more robust to the irregular spacing between meter digits. Given the model size constraint, collapsing two networks into one also helped with the memory budget. This paper documents the implementation, training, and evaluation of that system. The model reaches production-grade synthetic accuracy (validation loss 1.14 after 30 epochs on 100,000 images) but fails almost entirely on real photographs until fine-tuning nudges accuracy from zero to 6.5%, with night images standing out at 22.2%. We frame this not as a failure but as an instructive case study: the architecture is correct, the training pipeline is functional, and the failure is cleanly attributable to a single well-understood cause insufficient labelled real data. Fixing that cause is a matter of annotation effort, not of redesigning the model.

2. RELATED WORK

The sequence recognition approach at the core of this project derives directly from Shi et al. [1], who proposed the Convolutional Recurrent Neural Network (CRNN) as a unified architecture for image-based sequence recognition. Their key insight was that CNN feature extraction followed by a recurrent layer operating over the resulting feature columns could produce a sequence of character predictions without ever segmenting individual characters. Coupled with CTC loss [2], this allows training purely from word- or sequence-level labels, which is precisely what meter reading requires: the target is a six- or seven-digit string, not a series of individually labelled character crops. CTC, introduced by Graves et al. [2] in the context of speech recognition, solves the alignment problem between an output sequence and a target sequence of different lengths. The key mechanism is the introduction of a blank token and the marginalization over all valid alignments. One detail that caused significant trouble in our implementation: the CTC loss function expects raw logits, not SoftMax probabilities. Applying SoftMax before computing CTC loss causes the blank-token probability to dominate, resulting in trivially low entropy in the output distribution and

catastrophically large loss values (we observed values around 148 in early experiments). This is documented in the TensorFlow source but not prominently in most tutorials, and it took several debugging sessions to identify. The use of synthetic data to train OCR systems is well established, and models trained on synthetic data have been shown to generalize well to real images provided the synthetic distribution is close enough to the real domain. When it is not, the result is precisely what we observed: a model that learns the right output structure (sequences of digits) but cannot map that structure to the visual features of real photographs. Tobin et al. [3] coined the term "domain randomization" for the strategy of randomizing synthetic scene parameters so broadly that the real world appears as just another sample from the training distribution. Our Phase 2 synthetic data (100,000 images with varied fonts, noise patterns, and simulated lighting effects) was an attempt at this, but it could not fully replicate the appearance of real meter photographs under glass, outdoors, at varying angles. Automated utility meter reading has attracted growing attention as a computer vision application. Nguyen et al. [4] demonstrated a structured pipeline combining detection and recognition for water meters in varied environmental conditions, and Shen et al. proposed a deep learning approach to dial-type water meter recognition. These systems typically require large labelled datasets of real meter images hundreds to thousands of examples per condition which underscores the difficulty of the task with the 35 labelled images available in this project.

3. DATASET

We collected 170 photographs of analog utility meters using a smartphone camera under four distinct imaging conditions: standard daylight (with ambient reflections on meter glass), night with built-in LED illumination, artificially blurred images simulating motion or focus error, and intentionally tilted shots simulating imperfect camera mounting. Photographs were collected from a single physical meter across multiple days and lighting conditions, which limits the diversity of meter fonts and digit styles represented, but provides realistic variation in imaging artifacts. Of the 170 photographs, 35 were manually labelled with ground-truth meter readings and stored in a JSON file (real_image_labels.json). Four images were corrupted or missing at evaluation time, leaving 31 usable labelled examples: 9 daylight, 9 night, 5 blurred, and 8 tilted.

Why only 35 images were labelled? Manual labelling is straightforward for meter images one simply reads the displayed number but time-consuming when done carefully, particularly for blurred and low-light images where the reading is ambiguous. A complete label for our system requires not just the numeric reading but also the condition category and verification against a second reading. Given project time constraints, 35 images represented the maximum feasible effort. This limitation, known in advance, was the primary justification for the synthetic data strategy. The first synthetic dataset (phase 1) comprised 10,000 images rendered with simple white-on-dark backgrounds, clean fonts, and minimal variation. These were generated quickly and used for initial model validation. The model trained on this data reached a validation loss of 0.0003 essentially memorizing the domain but produced 0% accuracy on every real image tested. This result was informative; it confirmed that the model architecture was functional and that the synthetic distribution was simply too narrow to generalize. The second dataset (phase 2) comprised 100,000 images generated over approximately 41 hours, featuring varied fonts (including 7-segment display simulations), random noise overlays, simulated glare patches, varied contrast levels, and backgrounds sampled from real-world textures. The dataset contained approximately 99,792 unique numeric labels. The character vocabulary was fixed to the string "0123456789.-", yielding 12 characters plus one CTC blank token, for a total of 13 output classes. This dataset formed the primary training set and produced a model with validation

loss 1.14 substantially better than Phase 1, though as subsequent tests would show, still insufficient for direct deployment on real images.

4. METHODOLOGY

The core of our system is a CRNN in the style of Shi et al. [1], adapted for the specific constraints of meter reading: images of fixed aspect ratio (cropped to 200×800 pixels in grayscale) and output sequences of 5–8 characters drawn from a 12-character vocabulary. The network passes an input image through three convolutional blocks, reshapes the resulting feature map into a sequence of column vectors, and processes that sequence through two stacked bidirectional LSTM layers before projecting to per-timestep character logits. Table 1 shows the layer-by-layer configuration.

TABLE 1 - Model architecture summary. Total trainable parameters: 661,455 (~2.52 MB unquantized).

Layer	Configuration	Output Shape
Input	Grayscale	200 × 800 × 1
Conv2D + MaxPool	32 filters, 3×3, ReLU	100 × 400 × 32
Conv2D + MaxPool	64 filters, 3×3, ReLU	50 × 200 × 64
Conv2D + MaxPool	128 filters, 3×3, ReLU	25 × 100 × 128
Reshape	—	100 × 3200
Dense projection	256 units	100 × 256
Bi-LSTM	128 units per direction	100 × 256
Bi-LSTM	64 units per direction	100 × 128
Dense (output)	13 units, raw logits	100 × 13

The reshape operation is the key bridge between the spatial and sequential domains. After the third convolutional block, the feature map has shape 25×100×128. We collapse the height and channel dimensions, producing a sequence of 100 column vectors each of dimension 3,200, then project through a dense layer to 256 dimensions. The resulting 100-timestep sequence feeds into the recurrent layers. This design treats each horizontal column of the feature map as a "timestep" a natural fit for left-to-right reading of a horizontal digit string. The output layer produces raw logits over 13 classes (12 characters plus blank) at each of the 100 timesteps. It is critical that no SoftMax activation is applied at this stage. TensorFlow's `ctc_batch_cost` function expects log-probabilities that it computes internally from the logits; applying SoftMax beforehand flattens the distribution and causes the loss function to return values near 148 in our experiments a non-obvious failure mode that wasted several training runs. The fix is simply removing the final SoftMax activation, but diagnosing it required careful reading of the TensorFlow CTC source code. The full model was trained on 100,000 synthetic images using the Adam optimizer with a learning rate of 1×10^{-3} and a batch size of 32. Training ran for 30 epochs with early stopping based on validation loss. Loss fell from approximately 18.9 at epoch 1 to 1.14 at epoch 30. A second implementation issue arose from a Keras API change: the LSTM method `reset_states()` was renamed to `reset_state()` in newer Keras versions, causing an `AttributeError` that required updating the training loop.

Following synthetic training, we attempted to adapt the model to real images through supervised fine-tuning. The first three convolutional layers were frozen to preserve low-level feature detectors, while the dense projection, both LSTM layers, and the output layer remained trainable. The 28

training images (out of 35 labelled, with 7 reserved for validation) were augmented five times each through random brightness jitter, contrast scaling, rotation within ± 5 degrees, and horizontal shift, producing 140 effective training samples. Training used a reduced learning rate of 1×10^{-4} and ran for 73 epochs before early stopping triggered. The best checkpoint (by validation loss) was saved separately from the original synthetic-trained model, ensuring that the original weights were never overwritten. The current model weighs 2.52 MB in unquantized Keras format, marginally exceeding the 2 MB flash target. Post-training INT8 quantization, which was not applied in this project but is straightforward with TensorFlow Lite, would reduce this to approximately 0.7–0.9 MB and simultaneously enable faster inference on microcontrollers without a floating-point unit. The CRNN architecture, once converted to TFLite, is well within the capability of ESP32-S3 or similar inference-capable hardware; the ESP32-C6 without an FPU would require careful optimization, but the path is clear.

5. EXPERIMENTS AND RESULTS

Phase 1 training on 10,000 simple images produced a validation loss of 0.0003, an almost deceptively good result. When the Phase 1 model was applied to any of the 31 labelled real images, it produced 0 correct predictions. This established the baseline: near-perfect performance in a narrow synthetic domain translates to complete failure when that domain does not match reality. Phase 2 training on 100,000 realistic synthetic images told a more interesting story. Validation loss at epoch 30 reached 1.14, and the model generated structurally plausible predictions on real images five or six-digit numeric strings but the digit values themselves bore no consistent relationship to the ground-truth readings. Fine-tuning showed a gradual but limited improvement. Training loss fell from approximately 30.6 at epoch 1 to 8.75 at epoch 58 (the early-stopping point), while validation loss decreased from 27.8 to 13.80. The persistent gap between training and validation loss approximately 4.0 at the best checkpoint is a clear signal of overfitting. With 140 augmented training images spread across four imaging conditions, the model does not have enough signal to generalize. Table 2 shows accuracy broken down by imaging condition after fine-tuning.

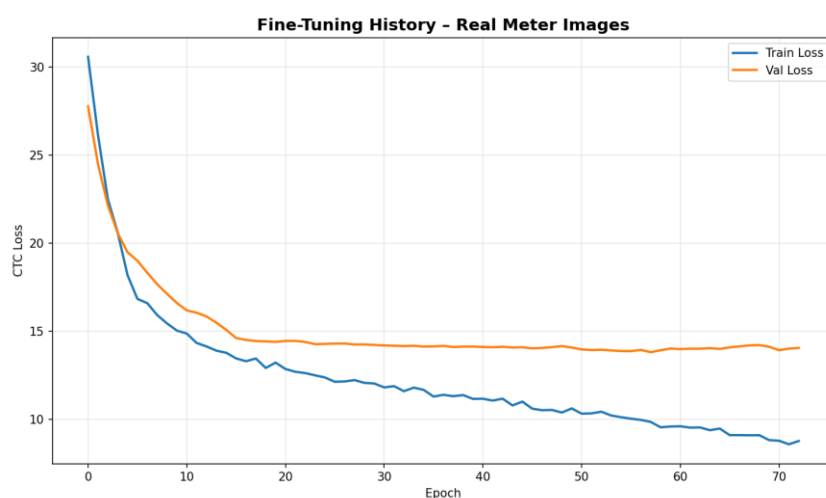


Figure 1 - shows the characteristic pattern of continued training loss decrease while validation loss flattens and diverges slightly, consistent with memorization of the small training set.

TABLE 2 - Per-condition accuracy on 31 labelled real images after fine-tuning. Pre-fine-tuning accuracy was 0% across all conditions.

Condition	Correct	Total	Accuracy
Day Light	0	9	0.0%
Night	2	9	22.2%
Blurred	0	5	0.0%
Tilted	0	8	0.0%
Overall	2	31	6.5%

The two correct predictions were both from the night condition: image 55.JPG (ground truth 135104, prediction 135104) and image 71.JPG (ground truth 019746, prediction 019746). Beyond exact matches, several images showed partial correctness that suggests genuine partial learning rather than random digit generation. Image 142.JPG had 5 of 6 digits correct; image 158.JPG had the first 3 digits correct; image 83.JPG had the final 3 digits correct; image 157.JPG had the last 4 digits correct. These partial matches are consistent with the CTC decoder correctly identifying some digit regions while misaligning others a positional rather than a perceptual failure.

6. DISCUSSION

The striking difference between night and daytime performance 22.2% versus 0.0% has a plausible explanation rooted in distribution alignment. Night images captured with LED illumination tend to show high-contrast white or amber digits against a dark background, a visual signature that closely matches our synthetic training images, which were generated with white digit masks on dark fields. Daylight images, by contrast, show complex ambient reflections on the glass meter face, lower effective contrast, and hue variations that the grayscale pipeline partially suppresses but cannot eliminate. The model was never exposed to this kind of visual complexity in training, and fine-tuning on 9 daytime images was too little to bridge the gap. The pattern of partial digit matches deserves attention. It would be tempting to dismiss 6.5% accuracy as "essentially zero," but the partial correctness evidence argues against this framing. A model outputting random five-digit strings would not consistently get the first three or last four digits right. The CTC decoding is performing genuine digit recognition, but the alignment between the predicted sequence and the ground truth is unstable the model identifies digits correctly but sometimes places them at the wrong horizontal position. This is expected behaviour when the training distribution provides insufficient examples of the full range of contrast and perspective variations present in the test set.

The path to useful accuracy is clear. The primary intervention required is more labelled real data. Based on comparable transfer learning projects in OCR and related domains, 200 to 300 labelled images per condition would likely produce acceptable fine-tuning results. A secondary improvement would be to integrate real images into the training pipeline from the start rather than treating fine-tuning as an afterthought: joint training on synthetic and real data with appropriately weighted losses has been shown to outperform sequential training in domain adaptation settings. Finally, a preprocessing stage that normalizes perspective (using detected meter frame corners) and equalizes contrast (CLAHE) before passing images to the CRNN would narrow the distribution gap without requiring additional labelled data. the current 2.52 MB Keras model slightly exceeds the 2 MB flash budget. This is not architecturally alarming. The LSTM layers account for the majority of the weight budget, and selective pruning of the recurrent weights or substitution of the bidirectional LSTMs with a lighter GRU variant could reduce the parameter count by 20–30% without major accuracy loss

on the synthetic domain. INT8 post-training quantization would then bring the TFLite model to well under 1MB. The deployment path exists; it simply requires the real-world accuracy.

7. CONCLUSION

This project set out to determine whether a CRNN+CTC model, trained on synthetic data and fine-tuned on real photographs, could form the basis of an ultra-lightweight meter reading system for deployment on resource-constrained hardware. The answer, based on our results, is: architecturally yes, practically not yet. The model learns the correct output structure, runs cleanly in Python and could be converted to TFLite, and improved on the real-image task from 0% to 6.5% accuracy with 35 training examples. That improvement, modest as it is, demonstrates that the fine-tuning approach is not fundamentally broken it is simply starved of data.

The project's most valuable output may be the clear identification of the binding constraint: labelled real images. Synthetic data can bootstrap a model to syntactic competence (learning what meter readings look like as sequences), but it cannot substitute for real perceptual experience. Future work should prioritize systematic collection and labelling of at least 200 images per condition, combined training from the start, and the application of INT8 quantization to bring the model within the 2 MB flash budget. With those ingredients, the target of 95%+ full-string accuracy appears achievable.

REFERENCES

- [1] Shi, B., Bai, X., & Yao, C. (2016). An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(11), 2298–2304.
- [2] Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006). Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 369–376.
- [3] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 23–30.
- [4] Nguyen, A., Tran-Trung, K., Huong, T. H., Hong, H. D. T., Trung, H. N., & Hoang, V. T. (2025). Water meter reading based on text recognition techniques and deep learning. *IEEE Access*, 13, 41422–41434.